CS540 Fall 2025 Homework 10

1 Assignment Goals

- Implement Value Iteration, a model-based dynamic programming algorithm for computing optimal policies.
- Implement Q-Learning, a model-free reinforcement learning algorithm that learns through interaction with the environment.
- Understand the difference between model-based and model-free approaches in reinforcement learning.
- Apply both algorithms to the FrozenLake-v1 environment and achieve target performance of average reward ≥ 0.7.
- Gain hands-on experience with the Gymnasium library for reinforcement learning environments.

2 Getting started

Download the starter code from Canvas. It consists of four main files: `q_learning.py`, `value_iteration.py`, `tests.py`, and `visualize.py`.

2.1 Environment Setup

Option 1: Using Conda (Recommended)

Create and activate a conda environment:

```
conda create -n hw10 python=3.10
conda activate hw10
pip install gymnasium==0.29.1 matplotlib==3.9.2 pygame==2.5.2 numpy
```

Option 2: Using venv

Create and activate a virtual environment:

```
python3 -m venv hw10_env
source hw10_env/bin/activate
pip install --upgrade pip
pip install gymnasium==0.29.1 matplotlib==3.9.2 pygame==2.5.2 numpy
```

Windows users: Replace `source hw10_env/bin/activate` with `hw10_env\Scripts\activate`

Note: When you open a new terminal, reactivate with `conda activate hw10` or `source hw10_env/bin/activate`.

To verify: `python -c "import gymnasium, matplotlib, numpy"`

Set up this environment on an instructional machine for final testing.

2.2 Gymnasium Environment Overview

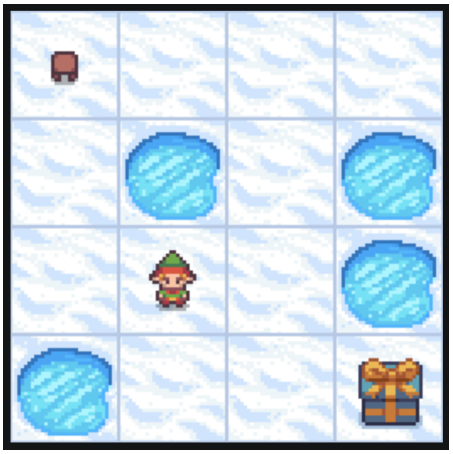

Figure 1: FrozenLake-v1 environment

You will use the FrozenLake-v1 environment from Gymnasium, a 4×4 grid where the agent navigates from start (top-left) to goal (bottom-right) while avoiding holes, as shown in Figure 1.

Environment properties:

- **Stochastic dynamics** (`is_slippery=True`): When you choose an action, the agent may slip and move in a perpendicular direction instead. For example, choosing RIGHT may result in the agent moving UP, RIGHT, or DOWN with equal probability (1/3 each).
- **Reward:** 1 for reaching goal, 0 otherwise
- **Episode termination:** Episode ends when reaching goal or falling into a hole

Key Gymnasium functions:

- **env.step(action)** — Takes action (0-3) and returns `(next_state, reward, terminated, truncated, info)`. Combine flags as `done = terminated or truncated`.
- **env.reset()** — Resets environment and returns `(initial_state, info)`.
- **env.action_space.sample()** — Returns random action (0-3).
- **env.action_space.n** — Number of actions (4: LEFT, DOWN, RIGHT, UP).
- **env.unwrapped.P[state][action]** — Returns transition dynamics as list of `(probability, next_state, reward, done)` tuples (needed for Value Iteration).

See `test.py` for examples of these function calls. For more details about the environment, visit: `https://gymnasium.farama.org/environments/toy_text/frozen_lake/`. Additional API documentation: `https://gymnasium.farama.org/api/env/`.

3 Value Iteration on FrozenLake-v1 (50 points)

For this part, you will implement Value Iteration on the FrozenLake-v1 environment described in Section 1. Value Iteration is a model-based algorithm that requires access to the environment's transition dynamics. The agent computes the optimal value function and derives the optimal policy without interacting with the environment through episodes.

3.1 Implementation

The main files for this part are `value_iteration.py`, and `test.py`. You will implement the Value Iteration algorithm in `value_iteration.py`.

Important: You only need to modify code within the TODO sections.

The Value Iteration algorithm iteratively updates the value function using the Bellman optimality equation:

$$V(s) \leftarrow \max_{a \in \mathcal{A}} \sum_{s',r} P(s', r|s, a) \left[r + \gamma V(s') \right] \tag{1}$$

where:

- $V(s)$ is the state value function
- $P(s', r|s, a)$ is the transition probability (accessible via `env.unwrapped.P`)
- γ is the discount factor
- The algorithm iterates until convergence (when $\max_s |V_{new}(s) - V_{old}(s)| < \theta$)

3.2 Algorithm Pseudocode

The core update step for each state can be implemented as follows:

Algorithm 1 Value Iteration Update for State s

$V_{old}(s) \leftarrow V(s)$
for each action $a \in \mathcal{A}$ **do**
  $R(s, a) \leftarrow 0$
  for each transition (p, s', r, done) from $P(s, a)$ **do**
    $R(s, a) \leftarrow R(s, a) + p \cdot [r + \gamma \cdot V(s') \cdot (1 - \text{done})]$
  end for
end for
$V(s) \leftarrow \max_a R(s, a)$
$\delta \leftarrow \max(\delta, |V(s) - V_{old}(s)|)$

Note on formulation: This pseudocode differs from the lecture slides (where the reward $r(s)$ appears outside the max operator) because in most RL environments, including FrozenLake-v1, the reward depends on the transition: $r(s, a, s')$. In the lecture formulation,

$$V(s) = r(s) + \gamma \max_a \sum_{s'} P(s'|s, a) V(s'),$$

the reward is assumed to depend only on the current state. However, in FrozenLake, the reward of 1 is only received when transitioning to the goal state, not simply for being in any particular state. Therefore, we use the more general Bellman equation:

$$V(s) = \max_a \sum_{s',r} P(s', r|s, a)[r + \gamma V(s')],$$

where the reward is inside the max operator and the expectation.

Convergence: You need to repeat this update for all states until $\delta < \theta$ (convergence threshold).

Once the value function converges, the optimal policy can be extracted by choosing actions that maximize the expected return at each state. Your code should save the learned V-table as `V_TABLE_ValueIteration.pkl`.

3.3 Submission

Submit the following files:

- `value_iteration.py`
- `V_TABLE_ValueIteration.pkl`

4 Q-Learning on FrozenLake-v1 (50 points)

For the Q-Learning portion, you will be using the same FrozenLake-v1 environment. Unlike Value Iteration, Q-Learning is a model-free algorithm that learns the optimal policy through trial and error by interacting with the environment. Q-Learning does not require knowledge of the transition dynamics and learns directly from experience.

4.1 Implementation

The main files for this part are `q_learning.py`, `tests.py`, and `visualize.py`. You will implement the Q-Learning algorithm in `q_learning.py`.

Important: You only need to modify code within the TODO sections.

For each sampled tuple $(s, a, r, s', \text{done})$, the Q-Learning update rule is:

$$Q(s,a) \leftarrow \begin{cases} (1-\alpha)Q(s,a) + \alpha\left(r + \gamma \max_{a' \in \mathcal{A}} Q(s', a')\right) & \text{if done = False} \\ (1-\alpha)Q(s,a) + \alpha r & \text{if done = True} \end{cases} \tag{2}$$

where:

- α is the learning rate
- γ is the discount factor
- The agent follows an ϵ-greedy policy as discussed in lecture

Once implemented, your code should save the learned Q-table as `Q_TABLE_QLearning.pkl`.

4.2 Submission

Submit the following files:

- `q_learning.py`
- `Q_TABLE_QLearning.pkl`

5 Information

test.py and visualize.py

The files `test.py` and `visualize.py` are provided to help you with the assignment. They do not need to be modified and should not be submitted.

test.py: This script evaluates the performance of your trained agent by running 100 episodes and computing the average reward. It is also useful for understanding how to interact with the Gymnasium environment. The script will automatically load your saved tables and display the results for both algorithms.

visualize.py: This script visualizes the learned policy as arrows on a grid, showing which action the agent takes in each state. The visualization helps verify that your agent has learned a reasonable policy, though it is not graded. Note that due to the stochastic nature of FrozenLake (`is_slippery=True`), the optimal policy may appear counterintuitive, prioritizing safety over the shortest path.

Optionally, you can try setting `gym.make("FrozenLake-v1", is_slippery=False)` to observe the difference in the learned policy. You will need to modify `q_learning`, `value_iteration`, and `visualize` files to do this.

However, please make sure to submit your code with `is_slippery=True` and ensure that `Q_TABLE_QLearning.pkl`and `V_TABLE_ValueIteration.pkl` correspond to `is_slippery=True`.

6 Submission

6.1 Due Date

The assignment is due Tuesday December 9 at 11:59pm Central Time. We are not accepting late submissions for this assignment.

6.2 Files

Submit the following files to Gradescope:

- `q_learning.py`
- `Q_TABLE_QLearning.pkl`
- `value_iteration.py`
- `V_TABLE_ValueIteration.pkl`

Test your implementations locally using `python test.py` with your virtual environment set up as specified in Section 1.

Do not submit: test.py, visualize.py, or any other extraneous files.

6.3 Grading

Gradescope will test on various performance thresholds. You should aim for an average episode reward of ≥ 0.7 on FrozenLake-v1 to achieve full points for each algorithm.

Grading is based on:

1. Correct implementation of the algorithms (Q-Learning and Value Iteration)
2. Performance of the learned/computed policy

Partial credit will be awarded based on the average reward achieved by your agent at different performance thresholds.